

Automated Affiliate Content Engine

1. Tujuan Project

Membangun aplikasi Python untuk mengotomatisasi proses pembuatan konten affiliate dari awal sampai siap dipublish.

Target awal:

- Menghasilkan konten affiliate secara otomatis.
- Menghasilkan script berdasarkan produk.
- Menghasilkan gambar/video.
- Menghasilkan voice-over.
- Menggabungkan asset menjadi video pendek 9:16.
- Menyimpan semua proses ke database.
- Menyediakan approval sebelum publish.
- Menyimpan statistik performa konten.
- Menggunakan data performa untuk membantu AI membuat konten berikutnya.
- Sistem harus modular sehingga provider AI, TTS, image provider, dan social media dapat diganti.

Target bisnis awal:

- Mendapatkan sekitar Rp10.000/hari dari affiliate.
- Fokus pada konsistensi dan otomatisasi, bukan mengejar viral.
- Hindari spam dan konten massal berkualitas rendah.

2. Tech Stack

Backend

- Python 3.12+
- FastAPI
- SQLAlchemy
- Alembic
- PostgreSQL
- Pydantic Settings

Task / Automation

Untuk MVP:

- APScheduler

Untuk scale-up:

- Redis
- Celery

Scheduler harus memanggil service/job, bukan memasukkan business logic langsung ke scheduler.

AI

Buat abstraction/provider interface.

Provider awal:

- OpenRouter
- Ollama

Contoh:

```
AIPProvider
├─ OpenRouterProvider
└─ OllamaProvider
```

Jangan mengikat business logic langsung ke OpenRouter.

Video

- FFmpeg
- Python subprocess untuk menjalankan FFmpeg
- Pillow untuk image processing jika diperlukan

Hindari ketergantungan berat pada library video Python jika FFmpeg sudah cukup.

TTS

Buat abstraction:

```
TTSPProvider
├─ LocalTTSPProvider
└─ CloudTTSPProvider
```

Provider dapat diganti melalui configuration.

Storage

MVP:

```
storage/
├─ images/
├─ audio/
└─ videos/
```

```
├─ thumbnails/
├─ temp/
```

Nanti dapat diganti menjadi:

- S3
- Google Cloud Storage
- MinIO

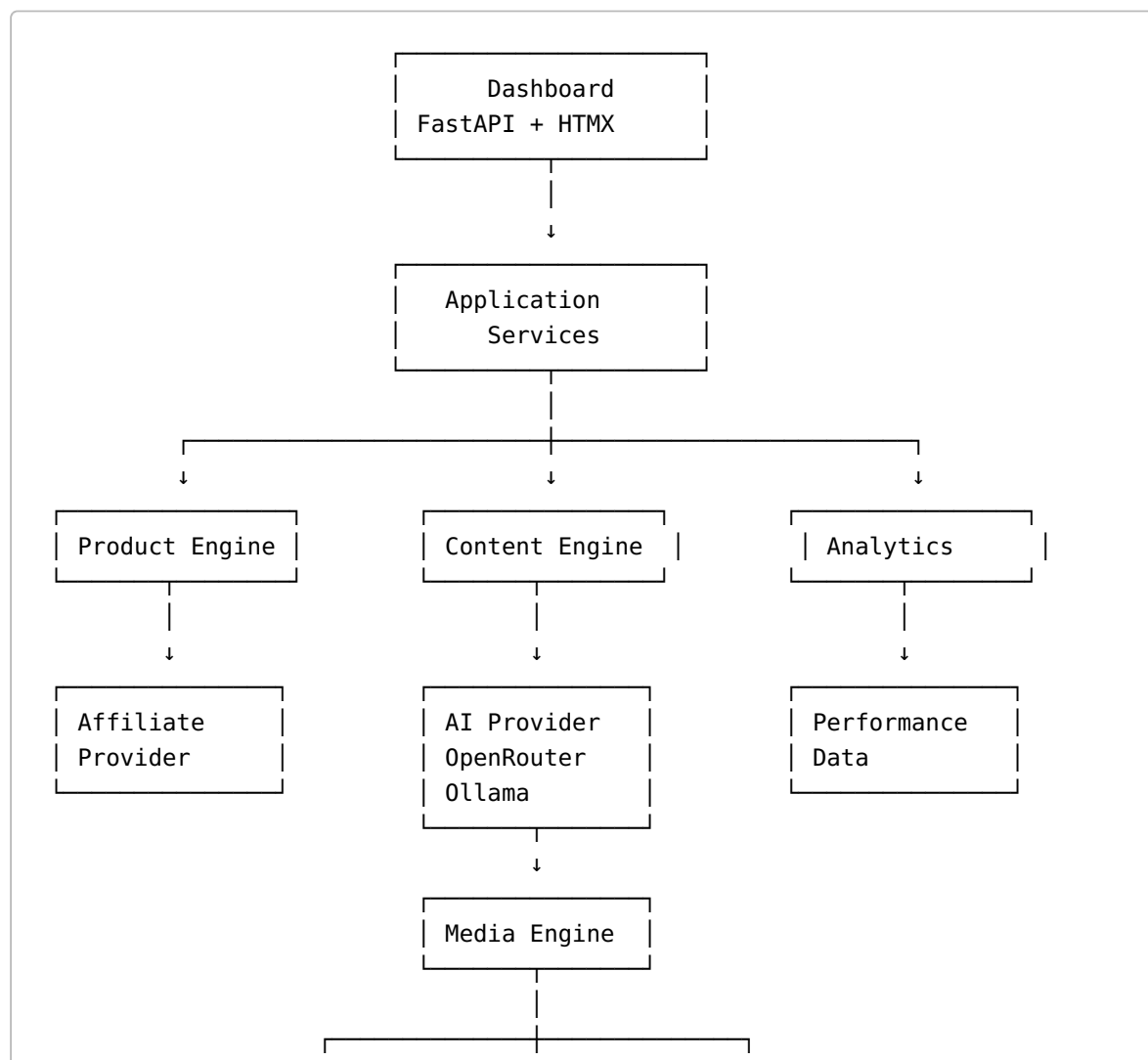
Frontend

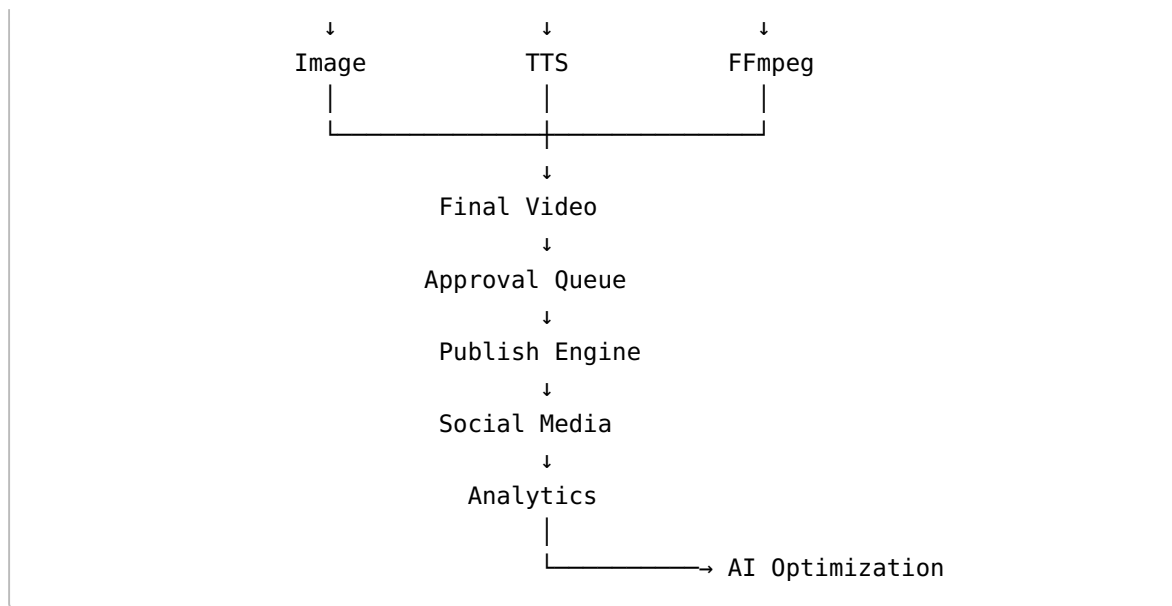
MVP dapat menggunakan:

- Jinja2 + HTMX

Tidak perlu SPA terlebih dahulu.

3. Arsitektur Besar





4. Main Workflow

Workflow A — Product Discovery

```
Scheduler
  ↓
Product Discovery
  ↓
Get products
  ↓
Normalize product data
  ↓
Calculate product score
  ↓
Save PostgreSQL
```

Product minimal data:

```
id
name
description
price
original_price
affiliate_url
image_url
category
commission
rating
sales_count
```

```
source
status
created_at
updated_at
```

Jangan melakukan scraping website yang melanggar Terms of Service.

Buat provider interface:

```
AffiliateProvider
```

Contoh provider:

```
AffiliateProvider
├─ ManualProductProvider
├─ ShopeeProvider
├─ TikTokProvider
└─ OtherProvider
```

Provider hanya digunakan jika API/integrasi resmi dan sesuai aturan platform tersedia.

5. Product Scoring

Setiap produk diberi score.

Contoh:

```
product_score =
    sales_score
    + rating_score
    + commission_score
    + price_score
    + content_potential_score
```

Tujuannya bukan mencari produk paling mahal.

Cari produk yang:

- mudah dijelaskan
- memiliki masalah yang bisa diselesaikan
- harga relatif terjangkau
- memiliki potensi impulse buying
- memiliki komisi masuk akal
- memiliki asset yang dapat digunakan secara legal

6. Workflow B — Generate Content Idea

```
Product
↓
AI Content Planner
↓
Generate multiple ideas
↓
Score ideas
↓
Select best idea
```

Contoh output:

```
{
  "hook": "...",
  "angle": "...",
  "target_audience": "...",
  "content_type": "problem_solution",
  "estimated_duration": 30
}
```

Jenis konten:

```
problem_solution
product_demo
comparison
top_list
before_after
tips
review
faq
```

AI harus menghindari klaim produk yang tidak diketahui kebenarannya.

7. Workflow C — Generate Script

Input:

```
Product
Content Idea
```

Target Audience
Duration

Output:

Hook
↓
Problem
↓
Solution
↓
Product Introduction
↓
Benefits
↓
Call To Action

Contoh struktur:

0-3 sec:
Hook

3-8 sec:
Masalah

8-20 sec:
Solusi / produk

20-25 sec:
Benefit

25-30 sec:
CTA

Script disimpan ke database.

Table:

content_scripts

Fields:

id
content_id
version
script_text

```
hook
cta
duration
ai_provider
ai_model
status
created_at
```

8. Workflow D — Generate Visual

Sistem harus mendukung dua mode.

Mode 1 — Existing Asset

Menggunakan gambar/video produk yang memang boleh digunakan.

Mode 2 — AI Generated

Generate visual pendukung berdasarkan script.

Buat abstraction:

```
ImageProvider
```

Contoh:

```
ImageProvider
├─ PexelsProvider
├─ LocalImageProvider
└─ AIImageProvider
```

Jangan mengasumsikan gambar dari internet bebas digunakan.

Simpan metadata source dan license/usage information jika tersedia.

9. Workflow E — TTS

```
Script
  ↓
TTS Provider
  ↓
Audio
```

↓
Save audio file

Database:

audio_assets

Fields:

id
content_id
provider
voice
language
file_path
duration
status
created_at

Bahasa awal:

id-ID

10. Workflow F — Video Generation

Input:

Script
Images / Video
Audio
Subtitle

Output:

1080x1920
9:16
MP4

Pipeline:

Assets
↓
Resize / Crop
↓
Timeline
↓
Voice
↓
Subtitle
↓
Background music
↓
CTA
↓
FFmpeg
↓
Final MP4

Setiap video memiliki status:

draft
processing
ready
approved
published
failed

11. Subtitle

Generate subtitle otomatis dari script/audio.

Format internal:

SRT

Contoh:

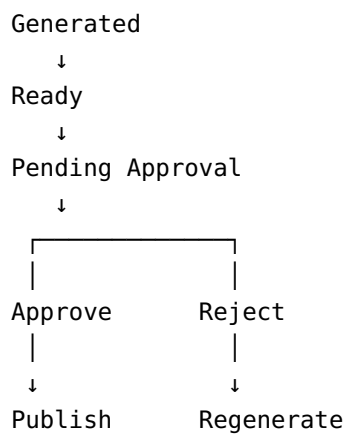
```
00:00:00,000 --> 00:00:03,000  
Kalau kamu sering mengalami masalah ini...  
  
00:00:03,000 --> 00:00:07,000  
Ada solusi sederhana yang bisa dicoba.
```

Subtitle harus readable pada video mobile.

12. Approval System

Jangan langsung publish otomatis pada MVP.

Flow:



Dashboard:

```
Content #001

Product:
...

Hook:
...

Script:
...

Video:
[Preview]

[ APPROVE ]
[ REJECT ]
[ REGENERATE ]
```

13. Publishing Engine

Buat abstraction:

```
SocialMediaProvider
```

Contoh:

```
SocialMediaProvider
├─ ManualExportProvider
├─ YouTubeProvider
├─ FacebookProvider
├─ InstagramProvider
└─ TikTokProvider
```

MVP:

```
Generate video
↓
Download/export
↓
Manual upload
```

Setelah workflow stabil, baru implement API publishing resmi.

Jangan menggunakan metode login automation yang melanggar Terms of Service platform.

14. Affiliate Link

Setiap content harus memiliki affiliate URL.

Database:

```
affiliate_links
```

Fields:

```
id
product_id
platform
original_url
affiliate_url
status
created_at
```

CTA video tidak boleh menampilkan link panjang.

Contoh:

```
"Cek link produk di bio."
```

atau CTA yang sesuai platform.

15. Analytics

Simpan:

```
views  
likes  
comments  
shares  
clicks  
sales  
commission
```

Table:

```
content_metrics
```

Contoh:

```
content_id  
platform  
views  
likes  
comments  
shares  
clicks  
sales  
revenue  
recorded_at
```

16. AI Optimization

Setelah data cukup:

```
Analytics  
↓
```

```
AI Analyzer
↓
Identify winning patterns
↓
Generate recommendations
↓
Content Planner
```

Contoh:

```
Video A
10.000 views
3 sales

Video B
1.000 views
0 sales

Video C
8.000 views
5 sales
```

AI dapat menyimpulkan:

```
Format problem_solution memiliki conversion
lebih baik daripada generic product introduction.

Hook dengan pertanyaan menghasilkan CTR
lebih tinggi.

Produk harga Rp20.000-Rp50.000 memiliki
conversion lebih baik.
```

Kemudian sistem membuat ide baru berdasarkan pola tersebut.

17. Database

Gunakan PostgreSQL.

Minimal table:

```
products
affiliate_links
content_ideas
contents
```

```
content_scripts
media_assets
audio_assets
videos
content_metrics
ai_generations
publish_jobs
settings
```

Relasi utama:

```
products
├── affiliate_links
└── contents
    ├── content_ideas
    ├── content_scripts
    ├── media_assets
    ├── audio_assets
    ├── videos
    ├── publish_jobs
    └── content_metrics
```

18. Status Machine

Content:

```
IDEA
↓
SCRIPT_GENERATED
↓
ASSETS_GENERATED
↓
VIDEO_GENERATED
↓
PENDING_APPROVAL
↓
APPROVED
↓
PUBLISHED
```

Jika error:

FAILED

dan bisa retry.

19. Project Structure

Gunakan struktur:

```
affiliate-content-engine/
|
├── app/
|   ├── main.py
|   ├── api/
|   │   ├── routes/
|   │   │   ├── products.py
|   │   │   ├── contents.py
|   │   │   ├── videos.py
|   │   │   ├── analytics.py
|   │   │   └── settings.py
|   │   └── dependencies.py
|   ├── core/
|   │   ├── config.py
|   │   ├── database.py
|   │   └── logging.py
|   ├── models/
|   │   ├── product.py
|   │   ├── content.py
|   │   ├── script.py
|   │   ├── media.py
|   │   ├── video.py
|   │   └── metrics.py
|   ├── schemas/
|   └── services/
|       ├── product_service.py
|       ├── content_service.py
|       ├── script_service.py
|       ├── media_service.py
|       ├── video_service.py
|       ├── analytics_service.py
|       └── publishing_service.py
|
```

```

├── providers/
│   ├── ai/
│   │   ├── base.py
│   │   ├── openrouter.py
│   │   └── ollama.py
│   ├── image/
│   │   ├── base.py
│   │   └── pexels.py
│   ├── tts/
│   │   └── base.py
│   ├── affiliate/
│   │   └── base.py
│   └── social/
│       └── base.py
├── jobs/
│   ├── discover_products.py
│   ├── generate_content.py
│   ├── generate_video.py
│   ├── publish_content.py
│   └── collect_metrics.py
├── templates/
│   ├── dashboard.html
│   ├── products.html
│   ├── contents.html
│   └── approval.html
├── utils/
│   ├── ffmpeg.py
│   ├── filesystem.py
│   └── text.py
├── alembic/
├── storage/
│   ├── images/
│   ├── audio/
│   ├── videos/
│   ├── thumbnails/
│   └── temp/
├── tests/
├── .env.example
├── .gitignore
└── requirements.txt

```

```
|─ alembic.ini
|─ docker-compose.yml
└─ README.md
```

20. Environment Variables

Contoh:

```
APP_NAME=Affiliate Content Engine
APP_ENV=development
APP_DEBUG=true

DATABASE_URL=postgresql+psycopg://user:password@localhost:5432/
affiliate_engine

OPENROUTER_API_KEY=
OPENROUTER_MODEL=

OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=

PEXELS_API_KEY=

STORAGE_PATH=./storage

FFMPEG_PATH=ffmpeg

DEFAULT_LANGUAGE=id-ID
DEFAULT_VIDEO_WIDTH=1080
DEFAULT_VIDEO_HEIGHT=1920
```

Secrets tidak boleh ditulis langsung di source code.

21. Scheduler

MVP:

```
08:00
Discover products

09:00
Generate content ideas

10:00
```

```
Generate scripts
```

```
11:00
```

```
Generate media
```

```
12:00
```

```
Generate videos
```

```
13:00
```

```
Add to approval queue
```

Jangan generate terlalu banyak sekaligus.

Gunakan configurable limit:

```
DAILY_PRODUCT_LIMIT=20
```

```
DAILY_CONTENT_LIMIT=5
```

```
DAILY_VIDEO_LIMIT=3
```

22. Queue

Setiap proses panjang harus bisa menjadi job.

Contoh:

```
GenerateVideoJob
```

```
GenerateAudioJob
```

```
GenerateImageJob
```

```
PublishJob
```

Job harus:

- memiliki status
- memiliki retry
- menyimpan error
- memiliki timeout
- idempotent

Jangan sampai retry membuat video atau publish menjadi duplikat.

23. Error Handling

Semua external provider harus memiliki:

```
timeout
retry
logging
rate limit handling
```

Contoh:

```
OpenRouter timeout
    ↓
retry
    ↓
failed
    ↓
save error
    ↓
dashboard
```

Jangan membuat infinite retry.

24. Logging

Gunakan structured logging.

Minimal:

```
timestamp
level
service
job_id
content_id
provider
message
error
```

Jangan menyimpan API key/token ke log.

25. Cost Control

Karena target awal hanya Rp10.000/hari, sistem harus mengutamakan biaya rendah.

Prioritas:

Local/Ollama

↓

Free API

↓

Low-cost API

↓

Paid API

AI provider harus dapat diganti melalui `.env`.

Jangan menggunakan AI mahal untuk semua tahap.

Contoh:

Ollama:

- classification
- scoring
- simple rewriting

OpenRouter:

- script generation
- content ideation
- analysis

26. MVP Phase

Phase 1

Buat:

FastAPI
PostgreSQL
SQLAlchemy
Alembic
Dashboard
Product CRUD

Belum perlu social media.

Phase 2

Tambahkan:

OpenRouter
Ollama
Content Idea Generator
Script Generator

Flow:

Product
↓
AI
↓
Content Idea
↓
Script

Phase 3

Tambahkan:

Pexels / image provider
TTS
FFmpeg
Subtitle

Flow:

Product
↓
Script
↓
Image
↓
Audio
↓
Subtitle
↓
Video

Phase 4

Tambahkan:

Approval Dashboard

Flow:

Video
↓
Preview
↓
Approve / Reject

Phase 5

Tambahkan:

Publishing API

Mulai dari platform yang memiliki API resmi dan akses yang tersedia.

Phase 6

Tambahkan:

Analytics

Phase 7

Tambahkan:

AI Optimization

Flow:

Analytics
↓
AI
↓
Winning Pattern
↓
New Content Ideas



New Videos

27. Prinsip Penting

1. Semua provider harus menggunakan interface/abstraction.
2. Jangan hard-code API provider.
3. Semua API key menggunakan environment variable.
4. Semua proses berat harus dapat dijalankan sebagai job.
5. Semua job harus dapat di-retry.
6. Jangan melakukan publish otomatis pada MVP.
7. Jangan melakukan scraping yang melanggar Terms of Service.
8. Jangan menggunakan asset yang tidak memiliki izin penggunaan.
9. Jangan membuat klaim produk yang tidak dapat diverifikasi.
10. Hindari spam dan duplicate content.
11. Setiap generated asset harus memiliki source metadata.
12. Semua content memiliki status lifecycle.
13. Semua proses penting harus tercatat di database.
14. Sistem harus dapat dijalankan tanpa provider tertentu.
15. Prioritaskan biaya rendah.

28. Definition of Done MVP

MVP dianggap berhasil apabila:

1. User memasukkan produk
- ↓
2. AI menghasilkan ide
- ↓
3. AI menghasilkan script
- ↓
4. Sistem mendapatkan asset
- ↓
5. Sistem menghasilkan voice
- ↓
6. FFmpeg menghasilkan video 9:16
- ↓
7. Video muncul di dashboard
- ↓
8. User dapat preview
- ↓
9. User dapat approve/reject
- ↓
10. Video dapat diekspor

Setelah flow ini stabil, baru tambahkan publishing otomatis dan analytics.

29. Prioritas Pengembangan

Jangan membuat seluruh fitur sekaligus.

Urutan:

```
[1] Project setup
    ↓
[2] PostgreSQL
    ↓
[3] Product CRUD
    ↓
[4] AI Provider
    ↓
[5] Script Generator
    ↓
[6] Image Provider
    ↓
[7] TTS
    ↓
[8] FFmpeg Video Generator
    ↓
[9] Approval Dashboard
    ↓
[10] Scheduler
    ↓
[11] Queue
    ↓
[12] Publishing
    ↓
[13] Analytics
    ↓
[14] AI Optimization
```

Jangan mengimplementasikan fitur Phase berikutnya sebelum Phase sebelumnya dapat berjalan.

30. Instruksi untuk AI Coding Agent

Mulai project ini dari Phase 1.

Sebelum coding:

1. Periksa environment Python.

2. Buat virtual environment.
3. Buat project structure.
4. Buat `.env.example`.
5. Buat PostgreSQL configuration.
6. Buat SQLAlchemy models.
7. Buat Alembic migration.
8. Buat FastAPI application.
9. Buat Product CRUD.
10. Buat dashboard sederhana.
11. Buat tests untuk Product CRUD.
12. Buat README dengan cara menjalankan project.

Jangan langsung membuat semua provider dan automation.

Setelah Phase 1 selesai dan dapat dijalankan, lanjutkan ke Phase 2.

Setiap phase harus:

- dapat dijalankan
- memiliki migration jika database berubah
- memiliki test
- memiliki README/update dokumentasi
- tidak merusak phase sebelumnya

Gunakan clean architecture sederhana dan jangan over-engineering.

Prioritaskan kode yang mudah dipahami dan mudah diganti providernya.